

Production-Grade Medical Imaging AI Platform (End-to-End AI Engineering Roadmap)

This document explains a complete end-to-end AI engineering project focused on Chest X-ray Pneumonia Detection. The project is designed specifically to help build strong production AI engineering skills relevant for top technology companies.

1. Project Goal

Build a production-grade medical imaging AI system capable of:

- Training deep learning models
- Running real-time inference
- Serving predictions via APIs
- Monitoring production performance
- Logging failures and metrics
- Deploying updates without disrupting production
- Supporting model versioning and rollback

2. Problem Statement

Radiologists manually analyze thousands of chest X-rays daily. This creates workload, fatigue, and delays in diagnosis.

The AI system assists by:

- Detecting pneumonia
- Prioritizing risky scans
- Providing explainable predictions
- Enabling scalable deployment

3. Why This Project is Strong

This project demonstrates:

- Computer Vision
- Deep Learning
- MLOps
- Deployment Engineering
- Monitoring
- CI/CD
- Production AI
- Explainable AI

- API Development
- Infrastructure Thinking

4. Hardware Consideration

Your hardware:

- ASUS TUF F15
- RTX 3050 GPU
- 16GB RAM
- 512GB SSD + 5TB external SSD

This is sufficient for:

- ResNet18 training
- EfficientNet training
- FastAPI serving
- Docker deployment
- MLflow tracking
- Streamlit dashboards
- Monitoring tools

5. Initial Model Choice

Start with ResNet18 because:

- Lightweight
- Fast training
- Low VRAM requirement
- Stable baseline
- Easier debugging
- Fast inference

Later upgrades:

- EfficientNet-B0
- DenseNet121
- Vision Transformer
- Swin Transformer

6. Dataset

Primary dataset:

NIH ChestXray14

Alternative:

RSNA Pneumonia Detection Dataset

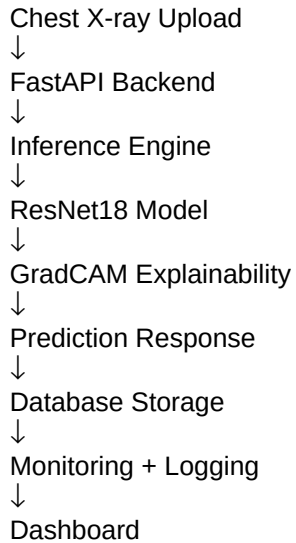
Initial task:

Binary classification:

- Normal

- Pneumonia

7. Full Production Architecture



8. Technology Stack

AI Layer:

- PyTorch
- torchvision
- timm

Backend:

- FastAPI
- Uvicorn
- Pydantic

Frontend:

- Streamlit

Database:

- PostgreSQL / Supabase

Experiment Tracking:

- MLflow
- Weights & Biases

Deployment:

- Docker
- GitHub Actions
- Render

Monitoring:

- Prometheus

- Grafana
- Sentry

9. Training Pipeline

Training Steps:

1. Load dataset
2. Preprocess images
3. Train model
4. Validate accuracy
5. Save checkpoints
6. Register best model
7. Push metrics to MLflow

10. Inference Pipeline

Production inference flow:

User Upload

↓

FastAPI API

↓

Image preprocessing

↓

Model inference

↓

GradCAM generation

↓

Prediction JSON response

11. Explainable AI

GradCAM highlights regions responsible for predictions.

This is important because:

- Medical AI requires transparency
- Helps build trust
- Improves debugging
- Enhances recruiter impact

12. CI/CD Pipeline

GitHub Actions workflow:

- Run tests
- Build Docker image

- Deploy backend
- Restart services
- Validate health checks

13. Monitoring & Observability

Metrics monitored:

- API latency
- Error rates
- GPU utilization
- Prediction confidence
- Request volume

Tools:

- Prometheus
- Grafana
- Sentry

14. Logging

Example production logs:

```
[INFO] Model loaded
[INFO] Prediction completed
[WARNING] Low confidence prediction
[ERROR] Image preprocessing failed
```

15. Real-Time Model Upgrades

One of the strongest features of this project:

You can deploy improved models without disrupting production.

Pipeline:

1. Train new model
2. Compare metrics
3. Register model in MLflow
4. Deploy new version
5. Route traffic gradually
6. Roll back if failures occur

16. Blue-Green / Canary Deployment Concept

Production-safe deployment strategy:

Old Model (v1)
↓
New Model (v2)
↓
Send small percentage of traffic
↓
Monitor metrics
↓
If stable:
Switch fully to v2

If issues:
Rollback to v1

17. Future Upgrades

After mastering the baseline:

- Multi-disease classification
- Vision Transformers
- Segmentation
- Medical report generation
- Federated learning
- Active learning
- Edge deployment

18. Skills Learned

By completing this project you learn:

- Deep learning engineering
- Real-time inference
- API development
- Docker
- CI/CD
- Monitoring
- Logging
- Model versioning
- MLOps
- Production deployment

19. Resume Impact

Strong resume statement:

Built and deployed a production-grade medical imaging AI platform for chest X-ray abnormality detection using PyTorch, FastAPI, Docker, MLflow, GitHub Actions, GradCAM explainability, Prometheus/Grafana monitoring, and automated model versioning pipelines.

20. Final Advice

Do not chase giant architectures initially.

Strong AI engineers:

- Start with stable baselines
- Build production systems
- Learn deployment
- Add monitoring
- Improve iteratively

The production pipeline is more valuable than starting with a huge model.